

PROJETO FINAL · VISÃO COMPUTACIONAL

Detecção e laudo automático de tumor cerebral

Um pipeline com Ultralytics YOLO26-L, dados sintéticos e diagnóstico sob orçamento de tempo fechado

Item	Descrição
Autor	Higor — Engenharia de Computação, EMC/UFG
Modelo	Ultralytics YOLO26-L (26,3 M parâmetros, cabeça end-to-end sem NMS)
Conjuntos	brain-tumor (1.116 imagens) · medical-pills (115, controle OOD) · 300 sintéticas
Ambiente	Google Colab · GPU T4 · Python 3 · ultralytics 8.4.x
Identidade visual	template Dtox (Themefisher): Bootstrap 4, Poppins, #008DEC e gradientes #17FFD3→#D3FC71
Restrição	execução completa em até 30 minutos
Entregáveis	notebook .ipynb · aplicação Flask · laudo JSON · painel HTML · este relatório

Aviso de escopo

Este documento descreve um exercício acadêmico de visão computacional. O sistema descrito **não é dispositivo médico**, não passou por validação clínica e **não deve ser usado para qualquer decisão diagnóstica**. Os rótulos **negative** e **positive** são convenção de anotação de um conjunto de dados público, não diagnóstico verificado por laudo clínico.

Sumário

1. Introdução e objetivo

O ponto de partida deste trabalho foi um notebook de aula que demonstra o uso do Ultralytics YOLO em imagem e vídeo com pesos pré-treinados. A pergunta que ele responde é «como se usa um detector pronto». A pergunta deste projeto é outra: **como se constrói um sistema inteiro em torno de um detector — e como se prova que ele funciona?**

O domínio escolhido é a detecção de lesões em cortes de ressonância magnética do encéfalo. A escolha não é acidental: é um domínio em que o custo de um falso negativo não é comparável ao de um falso positivo, em que a origem do dado (aparelho, protocolo) muda a distribuição de forma relevante e em que entregar uma caixa delimitadora não é entregar informação. Cada uma dessas três propriedades vira uma seção deste relatório.

O objetivo é produzir, dentro de um teto de trinta minutos de execução em GPU gratuita do Google Colab, um pipeline que: (i) particione o conjunto de dados de forma que a validação meça generalização real; (ii) amplie o conjunto de treino com dados sintéticos cujo rótulo seja exato por construção; (iii) treine um YOLO26-L com orçamento de tempo imposto; (iv) diagnostique o modelo em seis eixos independentes; e (v) transforme a saída em um laudo estruturado consumível por uma pessoa e por outro sistema.



Figura 1 — Arquitetura do pipeline. As etapas em amarelo são as que distinguem este trabalho de uma demonstração de detecção.

2. Fundamentação: o que muda no YOLO26

Três decisões de arquitetura do YOLO26 alteram diretamente o código escrito em torno dele, e por isso precisam ser explicitadas antes do método.

- **Cabeça end-to-end, sem NMS.** As versões anteriores emitiam centenas de candidatas e um pós-processamento por supressão de não-máximos escolhia as sobreviventes. O YOLO26 treina uma cabeça um-para-um, idealmente com uma caixa por objeto já na saída da rede. Consequência prática: os parâmetros `iou=` e `agnostic_nms=` perdem o efeito de antes.
- **Remoção da Distribution Focal Loss.** O DFL modelava cada coordenada como distribuição discreta sobre intervalos. Removê-lo simplifica a cabeça e preserva regressão sem faixa limitada, o que favorece alvos pequenos.
- **STAL e Progressive Loss.** A atribuição de rótulos ciente de alvos pequenos mantém cobertura positiva para objetos diminutos, e a perda progressiva desloca o peso do treino, ao longo das épocas, em direção à arquitetura que efetivamente roda na inferência.

Os dois primeiros pontos importam porque a mediana das caixas anotadas neste conjunto ocupa cerca de 1,5 % da área da imagem — território clássico de detecção de objetos pequenos. O terceiro importa porque uma consequência não documentada, verificada experimentalmente durante a construção deste projeto, é que **a cabeça end-to-end não aceita `augment=True`**: o Ultralytics emite `Model does not support 'augment=True'` e retorna à inferência em escala única. Isso elimina a rota mais comum de estimativa de incerteza por aumento em tempo de teste e obrigou o desenho de uma alternativa, descrita na seção 4.4.

3. Materiais

3.1 Conjuntos de dados

Os dois conjuntos utilizados são distribuídos pela Ultralytics sob licença AGPL-3.0 e baixam sem credencial, o que preserva a reprodutibilidade em qualquer máquina.

Conjunto	Papel no projeto	Imagens	Observação
brain-tumor	conjunto primário de detecção	1.116	escala de cinza, 2 classes, ~4,3 MB
medical-pills	controle negativo fora do domínio	115	nenhuma contém encéfalo
sintético	ampliação do treino	300	gerado no próprio notebook

Uma honestidade necessária sobre os rótulos: neste conjunto público, [negative](#) e [positive](#) são classes de caixa, não diagnósticos verificados. Praticamente toda imagem anotada possui uma caixa. Portanto, o que o modelo aprende é «localizar a região de interesse e classificá-la na convenção do conjunto», e não «diagnosticar tumor». Essa distinção reaparece na seção 7.

3.2 Particionamento por protocolo de aquisição

A leitura crua do conjunto — abrindo cada imagem e cada arquivo de rótulo — revela que ele não é homogêneo: convivem ao menos três geometrias dominantes, 512×512 , 192×256 e 256×256 . Em imagem médica, geometria distinta costuma indicar equipamento ou protocolo distinto, com outra relação sinal-ruído e outra estatística de intensidade.

O detalhe que define o desenho experimental é este: **as 175 imagens de 192×256 estão todas no split de treino original**, e o split de validação original não contém nenhuma delas. Validar no split original mede o modelo apenas em protocolos que ele já viu. Em vez de importar um segundo conjunto externo — o que traria dependência de credencial e consumiria minutos do orçamento —, este projeto extrai o segundo conjunto do próprio dado, usando a geometria como **proxy** de fonte.

Partição	Papel	Imagens	Caixas	negativas	positivas	Protocolo
treino	ajuste dos pesos	718	752	409	343	$512^2 \cdot 256^2 \cdot \text{raros}$
interno	validação in-distribution	223	241	154	87	$512^2 \cdot 256^2$
externo	validação sob deslocamento	175	173	28	145	192×256 exclusivo

A regra é dura de propósito: nenhuma imagem 192×256 entra no treino. Não é um holdout aleatório, é um **holdout de protocolo**. A tabela também expõe o achado mais interessante do projeto: no treino as classes estão quase equilibradas (46 % positivas), enquanto no conjunto externo **84 % das caixas são positivas**. Há,

portanto, deslocamento de covariável (a imagem mudou) *e* deslocamento de rótulo (a prevalência mudou) — dois problemas distintos, que cobram preços distintos.

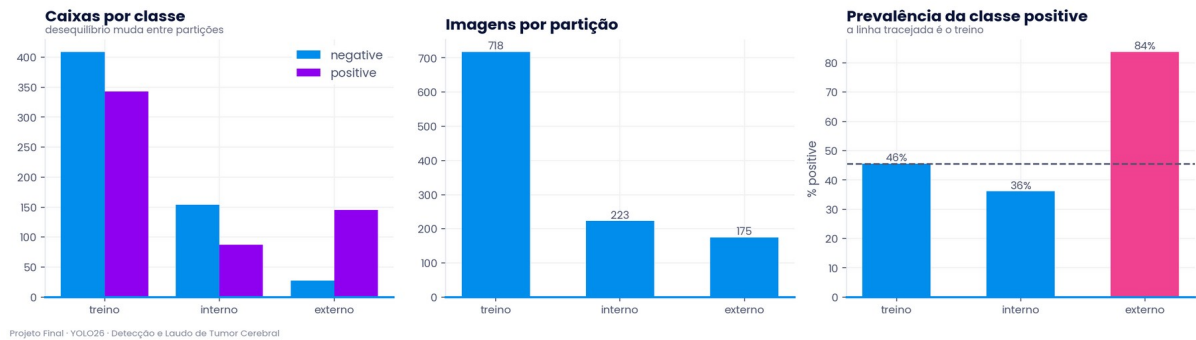


Figura 2 — Composição das partições. À direita, a prevalência da classe positiva; a linha tracejada marca o valor do treino.

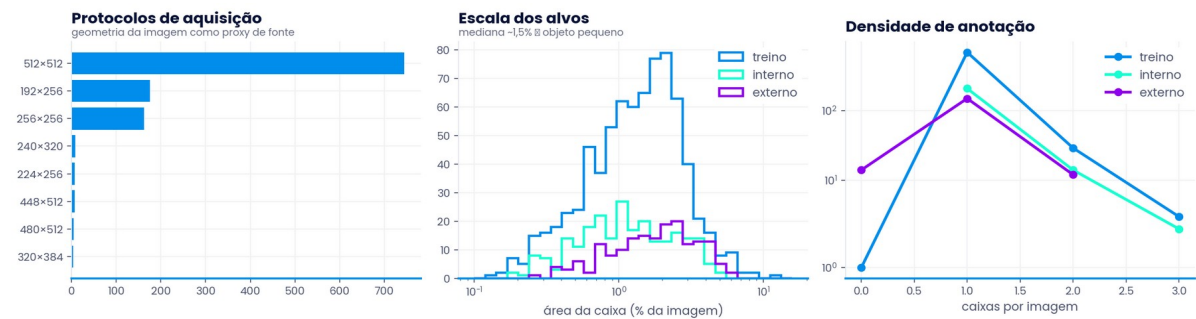


Figura 3 — Protocolos de aquisição, escala dos alvos e densidade de anotação por partição.

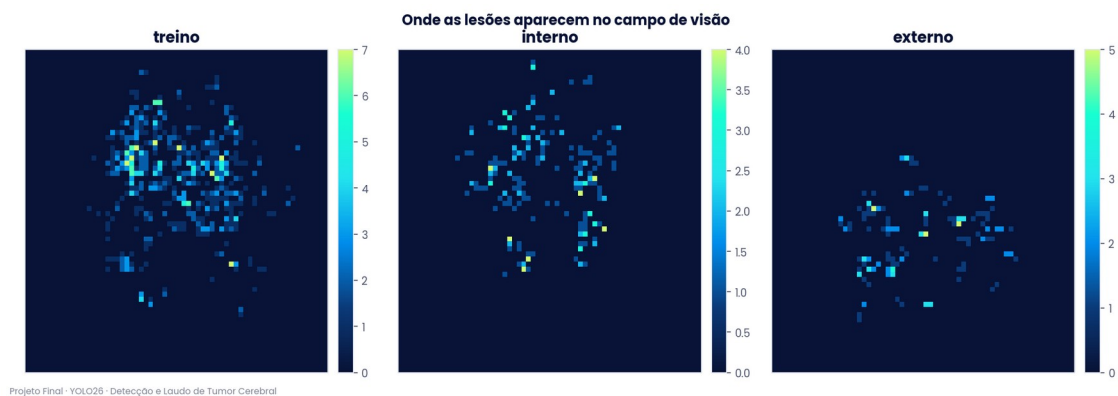


Figura 4 — Distribuição espacial dos centros das lesões em cada partição.

4. Método

4.1 Síntese de lesões com rótulo exato

O aumento de dados que o Ultralytics já executa (mosaico, HSV, escala, espelhamento) não cria lesões novas em posições novas — e é justamente isso que falta com 718 imagens de treino. A síntese adotada recorta caixas anotadas do próprio treino e as recola em outras imagens, com quatro cuidados que evitam que o detector aprenda a costura em vez da lesão.

- **Onde colar.** Máscara intracraniana obtida por limiarização de Otsu, maior componente conexo, fechamento morfológico e preenchimento de contorno; em seguida, erosão para afastar a colagem da calota craniana.
- **Como misturar.** Máscara elíptica com borda difusa: para a distância normalizada d do centro, $\alpha = \text{clip}((1 - d)/s, 0, 1)$ e $I = \alpha \cdot I_{\text{lesão}} + (1 - \alpha) \cdot I_{\text{fundo}}$.
- **Harmonizar intensidade.** O recorte vem de outra imagem, com outro brilho; casam-se os dois primeiros momentos com a vizinhança do destino, $I' = (I - \mu_{\text{lesão}})/\sigma_{\text{lesão}} \cdot \sigma_{\text{viz}} + \mu_{\text{viz}}$.
- **Alternativa por Poisson.** Em parte das amostras usa-se seamless cloning, que resolve $\nabla^2 f = \nabla^2 g$ no interior da região com $f = I_{\text{fundo}}$ no contorno; a costura desaparece porque a solução impõe continuidade na fronteira.

O ponto decisivo é que **o rótulo não é estimado, é conhecido**: a posição de colagem foi escolhida, logo a caixa é exata por construção. Dado sintético só é defensável quando o rótulo vem junto com o pixel.

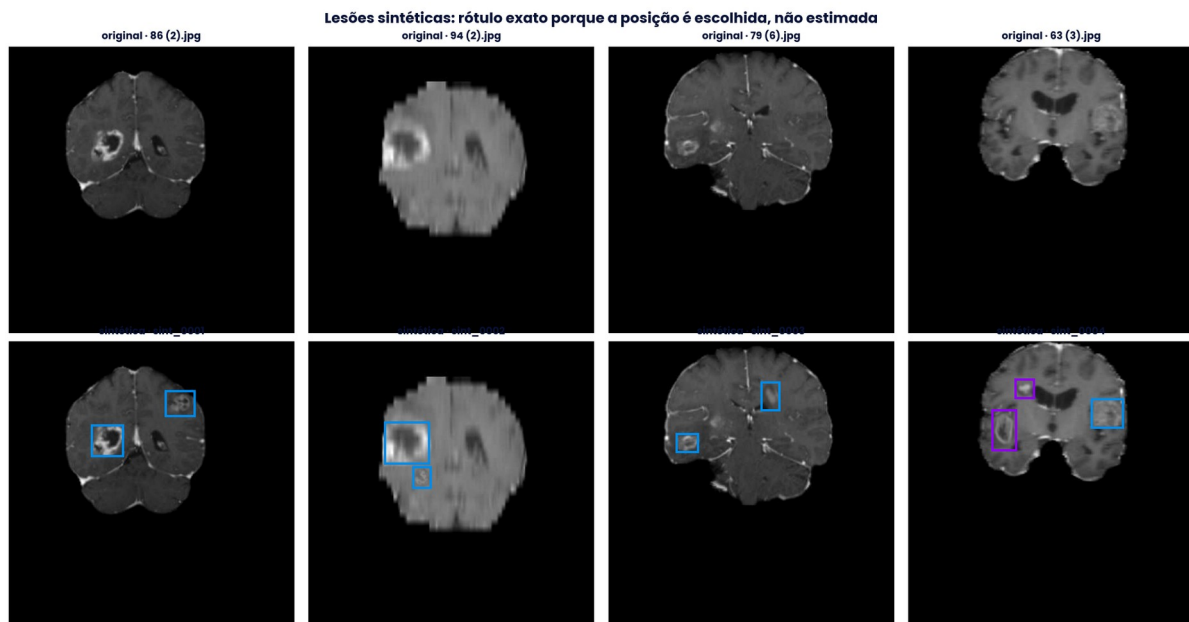


Figura 5 — Quatro exemplos de síntese: imagem original acima, versão sintética abaixo com as caixas resultantes.

4.2 Artefatos de ressonância para o teste de robustez

Somar ruído gaussiano a uma imagem de ressonância é fisicamente incorreto. A imagem de magnitude é o módulo de um sinal complexo cujos dois canais

carregam ruído gaussiano independente, de modo que a observação segue distribuição Rician, $I_{\text{obs}} = \sqrt{((I + n_1)^2 + n_2^2)}$. Quatro degradações foram implementadas a partir do modelo físico, três delas diretamente no espaço-k.

Artefato	Origem física	Simulação
ruído Riciano	ruído térmico no sinal complexo	módulo do sinal com ruído nos dois canais
campo de bias	não-uniformidade de B_1	multiplicação por campo polinomial suave
ghosting $N/2$	instabilidade entre linhas de codificação de fase	fase $e^{i\pi\lambda}$ em linhas alternadas do espaço-k
ringing de Gibbs	truncamento do espaço-k	zeragem das altas frequências e IFFT

Nenhuma dessas imagens entra no treino. Elas existem exclusivamente para o teste de robustez da seção 5.5. Misturar as duas famílias de síntese seria treinar e avaliar na mesma distribuição artificial — o erro metodológico mais comum em trabalhos com dado sintético.

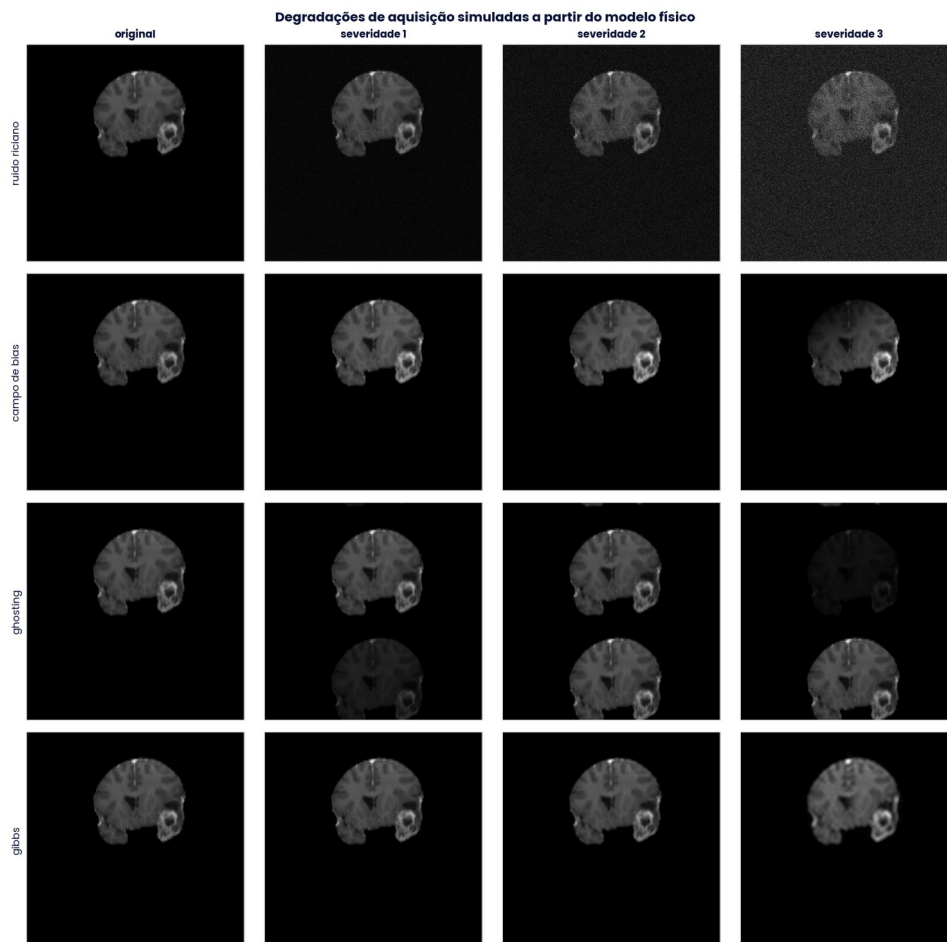


Figura 6 — Os quatro artefatos em três severidades, aplicados ao mesmo corte.

4.3 Treino sob orçamento de tempo imposto

O Colab sorteia o hardware: a mesma célula pode receber uma T4, uma L4 ou uma CPU, e a mesma configuração de épocas produz tempos que variam por um fator de cinco. Um notebook que promete trinta minutos com um número fixo de épocas não promete nada.

A solução adotada inverte a variável de controle. O Ultralytics aceita o argumento `time`, documentado como «tempo máximo de treinamento em horas; se definido, sobrescreve o argumento `epochs`, permitindo que o treinamento pare automaticamente após a duração especificada». O treinador mede a duração real da primeira época, recalcula quantas cabem no tempo e encerra devolvendo o melhor checkpoint. **O número de épocas passa a ser saída do experimento, não entrada**, e o relógio — que é o que o usuário sente — fica fixo. O comportamento foi verificado: com `epochs=2`, `time=0.05`, o treinador reprogramou a execução para 55 épocas.

Argumento	Valor	Justificativa
<code>time</code>	0,25 h (15 min)	impõe o teto; sobrescreve <code>epochs</code>
<code>epochs</code>	120	apenas um teto superior; quem decide é <code>time</code>
<code>imgsz</code>	640	compromisso entre alvo pequeno e velocidade
<code>cache</code>	ram	≈1.000 imagens pequenas; o gargalo é o disco do Colab
<code>cos_lr</code>	True	decaimento suave, robusto a número de épocas indeterminado
<code>close_mosaic</code>	5	o modelo vê imagens inteiras antes de encerrar
<code>deterministic</code>	False	reprodutibilidade total custaria velocidade — assunção declarada
<code>patience</code>	100	o critério de parada é o tempo, não a estagnação

4.4 Motor de laudo

A última etapa converte tensores em informação interpretável. Para cada achado acima do limiar operacional, o motor calcula área, diâmetro equivalente ($D = 2\sqrt{A/\pi}$), área relativa à região intracraniana, razão de aspecto, lateralidade em relação à linha média estimada, contraste relativo ao parênquima e um indicador de estabilidade. O resultado é um objeto JSON versionado, com treze campos por achado, mais um bloco de resumo e um aviso de escopo.

Como `augment=True` não é suportado (seção 2), a estabilidade é obtida repetindo a inferência com a imagem espelhada horizontalmente — devolvendo as caixas ao referencial original — e em outra escala de entrada, e medindo o IoU médio entre as detecções originais e as das duas vistas. **Esse indicador não é uma probabilidade calibrada**: é concordância entre vistas, e um modelo consistentemente errado exibe estabilidade alta.

Medidas físicas ausentes por decisão

Nenhuma medida é reportada em milímetros. Arquivos JPEG não carregam o campo `PixelSpacing` do DICOM, e converter pixel em milímetro sem esse cabeçalho seria o erro mais fácil e mais perigoso do projeto. O campo existe no esquema do laudo e permanece vazio até que a escala seja informada explicitamente.

5. Protocolo de avaliação

Relatar um valor de mAP e encerrar é o padrão dos trabalhos de disciplina, mas mAP é uma média de médias: informa quanto, nunca onde nem por quê. O protocolo adotado substitui um número por seis perguntas independentes.

#	Pergunta	Instrumento
5.1	Quão bem, no total?	mAP@50 e mAP@50-95 recalculados a partir das predições cruas
5.2	Com qual limiar operar?	varredura de τ , curva precisão-recall, duas políticas de escolha
5.3	A confiança reportada significa algo?	ECE e diagrama de confiabilidade
5.4	Que tipo de erro o modelo comete?	taxonomia inspirada no TIDE e matriz de confusão
5.5	Ele sobrevive a outro aparelho?	partição externa e artefatos simulados
5.6	Ele inventa achado onde não há nada?	controle negativo medical-pills

5.1 Métricas e casamento

As predições são coletadas com limiar baixo (0,05), porque as curvas precisão-recall precisam da cauda de baixa confiança, e passam por uma etapa explícita de consolidação por IoU — a cabeça end-to-end deveria tornar duplicatas raras, mas «deveria» não é garantia, e a consolidação torna o laudo determinístico. O casamento é guloso: as predições são ordenadas por confiança decrescente e cada uma disputa a caixa verdadeira de maior IoU ainda livre. A precisão interpolada em todos os pontos define $AP = \sum (r_{i+1} - r_i) \cdot p_{interp}(r_{i+1})$, e o mAP@50-95 repete o cálculo para dez limiares de IoU.

5.2 Escolha do ponto de operação

Um detector devolve pontuações, não decisões; a decisão nasce quando alguém escolhe um corte τ , e essa escolha é um julgamento de valor. Duas políticas foram implementadas: **** τ de F1, que maximiza a média harmônica entre precisão e recall, e τ de triagem****, o maior τ que ainda garante recall $\geq 0,85$. No contexto simulado, a segunda é a defensável, porque o custo de um falso negativo não é comparável ao de um falso positivo. Quando o recall alvo é inatingível, o notebook registra isso explicitamente e recai sobre τ^* .

5.3 Calibração

O Expected Calibration Error agrupa as detecções em M faixas de confiança e mede $ECE = \sum (|B_m|/n) \cdot |\text{acc}(B_m) - \text{conf}(B_m)|$. Importa porque o laudo imprime a confiança para uma pessoa ler, e um valor superconfiante impresso em um laudo é pior do que nenhum valor.

5.4 Taxonomia de erros

Dois modelos com o mesmo mAP podem errar de formas opostas, e as correções também são opostas. As categorias registradas são: classe errada com IoU adequado; caixa mal posicionada ($0,1 \leq \text{IoU} < 0,5$); classe e caixa erradas; duplicata; invenção sobre o fundo ($\text{IoU} < 0,1$); e lesão não detectada — o erro caro em triagem.

5.5 e 5.6 Robustez e alarme falso fora do domínio

A robustez é medida aplicando os quatro artefatos da seção 4.2 à partição interna, em três severidades, e observando a queda de $\text{mAP}@50$. O controle negativo utiliza as 115 imagens de medical-pills: nenhuma contém encéfalo, logo toda caixa emitida ali é invenção, e a fração de imagens com alarme é uma métrica de segurança direta — a frequência com que o sistema entregaria laudo positivo para uma entrada que jamais deveria ter sido processada.

6. Resultados

Como esta seção é preenchida

Este relatório **não afirma nenhum valor de desempenho**. As métricas dependem da GPU sorteada pelo Colab e do número de épocas que couberam no orçamento, e variam entre execuções. Ao final da execução o notebook grava `resultados.json`, que é a fonte única dos números abaixo; a tabela deve ser preenchida a partir desse arquivo, e o painel HTML do projeto lê o mesmo arquivo e desenha os gráficos automaticamente. Preencher à mão qualquer célula sem consultá-lo anula a rastreabilidade que o projeto inteiro foi desenhado para ter.

6.1 Quadro de resultados

Métrica	Chave em resultados.json	Valor
mAP@50 · partição interna	metricas.interno.map50	
mAP@50-95 · partição interna	metricas.interno.map50_95	
mAP@50 · partição externa	metricas.externo.map50	
mAP@50-95 · partição externa	metricas.externo.map50_95	
Queda de mAP@50 sob deslocamento (%)	metricas.queda_mAP50_dominio_pct	
AP@50 por classe (negative / positive)	metricas.ap_por_classe	
ECE	metricas.ece	
Limiar operacional τ adotado	metricas.tau_operacional	
Precisão e recall em τ (interno)	metricas.ponto_interno	
Épocas efetivamente concluídas	treino.epocas_concluidas	
Taxa de alarme falso fora do domínio	ood.taxa_alarme_falso	
Tempo total medido (min)	tempo.total_min	

6.2 Leitura esperada

Independentemente dos valores obtidos, três leituras devem constar da análise:

- **A diferença entre interno e externo é o número mais informativo do relatório.** Se houver queda relevante, a conclusão correta não é «o modelo é ruim», mas «o valor medido na partição interna superestimava a realidade» — que é exatamente o que a maioria dos trabalhos publica sem perceber.

- **A curva de métricas por época indica se o orçamento foi o gargalo.** Se ela ainda subia quando o relógio parou, o valor reportado é um piso, não o teto do modelo, e isso precisa ser dito.
- **A taxa de alarme falso fora do domínio é uma métrica de segurança, não de desempenho.** Um valor alto significa que o sistema entregaria laudo positivo para entradas que nunca deveriam ter chegado até ele, e aponta para a necessidade de uma verificação de domínio antes da detecção.

7. Limitações

A parte mais importante de um relatório técnico é a lista do que ele não sustenta.

- **Não é diagnóstico.** As classes são convenção de anotação de um conjunto público, sem laudo clínico verificado. O sistema aprende a convenção, não a doença.
- **A partição externa é um proxy.** Geometria de aquisição sugere fonte diferente, mas não prova. A afirmação defensável é «há deslocamento de distribuição mensurável entre esses subconjuntos», não «isto é validação multicêntrica».
- **Os dados sintéticos herdam o viés do original.** As lesões coladas vêm do mesmo banco de 745 recortes: ampliam variedade de posição e escala, não criam patologia que o conjunto não continha.
- **Os artefatos são simulados.** Reproduzem o modelo físico, não a distribuição empírica de um equipamento real.
- **A estabilidade não é probabilidade.** É concordância entre três vistas da mesma imagem.
- **O orçamento limita o resultado.** Com quinze minutos de treino, é plausível que as métricas ainda estivessem subindo quando o relógio parou.
- **Não há escala física.** Sem o campo PixelSpacing do DICOM, nenhuma medida é reportada em milímetros.

7.1 Trabalhos futuros

- Ablação com e sem dados sintéticos, medida na **partição externa** — a única comparação que responde se a síntese ajudou de fato.
- Substituir caixa por segmentação ([yolo26l-seg.pt](#)): área de lesão medida em pixels de máscara é substancialmente melhor que largura × altura.
- Calibração post-hoc por temperatura sobre as confianças, com reavaliação do ECE.
- Validação cruzada por protocolo (leave-one-protocol-out) em vez de um único holdout.
- Verificação de domínio na entrada, rejeitando imagens fora da distribuição antes da detecção.

8. Rastreabilidade de requisitos

Um entregável só é auditável se cada requisito apontar para a evidência que o satisfaz. A mesma matriz é gerada pelo notebook e exportada como [rastreabilidade.csv](#).

ID	Requisito	Seção do notebook	Evidência
R1	Partir do notebook-base e manter o estilo de aula	todas	estrutura markdown → código, seção a seção
R2	Usar YOLO26 na versão L no mínimo	§1 e §5	cfg.modelo = yolo26l.pt
R3	Incluir dados sintéticos	§4.1	300 imagens com rótulo exato por construção
R4	Incluir um segundo conjunto de dados	§3 e §3.1	partição externa (175 img) + medical-pills OOD (115 img)
R5	Pipeline com diagnóstico detalhado	§6	seis eixos independentes de avaliação
R6	Laudo estruturado por imagem	§7	JSON versionado, 13 campos por achado
R7	Interface web funcional	§8	Flask servindo o template Dtox: 4 páginas + API JSON
R8	Executável em até 30 minutos	§5 e §9.1	teto imposto via time=; tempo medido e reportado
R9	Projeto inteiro construído sobre o template	§0.3 e §8	paleta e tipografia do Dtox em todas as figuras e páginas

9. Reprodutibilidade e interface

9.0 Aplicação web

A interface não foi desenhada do zero: ela é o template **Dtox** (Themefisher) — Bootstrap 4, tipografia Poppins, o azul #008DEC e os gradientes #17FFD3 → #D3FC71 e #17FFD3 → #23E3EE — servido por **Flask**. A mesma paleta governa todas as figuras deste relatório, de modo que documento, notebook e aplicação formam um único sistema visual.

O back-end não reimplementa nada: as rotas chamam exatamente as funções `gerar_laudo`, `carregar_cinza` e `mascara_encefalo` já usadas no notebook. Uma interface que reimplementa a regra de negócio é uma segunda fonte de verdade esperando para divergir da primeira.

Rota	O que faz
/	apresenta o pipeline, as partições e a síntese, com as figuras da execução
/laudo	envia uma imagem, ajusta o limiar e devolve o laudo completo e o JSON
/resultados	lê o resultados.json da execução e monta o painel de diagnóstico
/metodo	método, decisões de treino, eixos de avaliação e limitações
/api/laudo	o mesmo motor em JSON, para integração com outro sistema

A aplicação roda em dois modos, detectados automaticamente: com o pacote completo do Dtox presente em `static/dtox`, usa o template inteiro (imagens de fundo, formas decorativas, AOS, slick); sem ele, usa um subconjunto do próprio `style.css` e do Bootstrap que acompanham o template, preservando o mesmo layout.

9.1 Como executar

Abrir o notebook no Google Colab, definir o acelerador de hardware como GPU e executar todas as células. O comportamento inteiro é controlado por um único objeto `Config` e por três perfis: `rapido` (YOLO26-S, 6 min de treino), `padrao` (YOLO26-L, 15 min — o perfil desta entrega) e `completo` (YOLO26-X, 45 min). Um quarto perfil, `teste`, executa o notebook inteiro em CPU apenas para verificar que nenhuma célula quebra.

Arquivo produzido	Conteúdo
fig/01...16_*.png	dezesseis figuras na paleta do projeto
runs/yolo26_<perfil>/weights/best.pt	pesos treinados
laudo_exemplo.json	laudo estruturado de uma imagem
laudos_lote.csv	tabela de laudos, ordenável por carga lesional
rastreabilidade.csv	matriz requisito → seção → evidência

Arquivo produzido	Conteúdo
resultados.json	todas as métricas da execução; fonte única deste relatório

O pacote [ultralytics](#) e os conjuntos brain-tumor e medical-pills são distribuídos sob AGPL-3.0; trabalhos derivados disponibilizados como serviço herdam essa obrigação. O template **Dtox** é da Themefisher e acompanha a sua própria licença, incluída no pacote da aplicação. Convém verificar ambas antes de qualquer uso além do acadêmico.

Referências

- REDMON, J. et al. You Only Look Once: Unified, Real-Time Object Detection. arXiv:1506.02640, 2015.
- ULTRALYTICS. YOLO26 — Unified Real-Time End-to-End Vision Models. Documentação oficial. Disponível em: docs.ultralytics.com/models/yolo26.
- ULTRALYTICS. Brain Tumor Dataset e Train Mode. Documentação oficial. Disponível em: docs.ultralytics.com.
- BOLYA, D. et al. TIDE: A General Toolbox for Identifying Object Detection Errors. ECCV, 2020.
- GUO, C. et al. On Calibration of Modern Neural Networks. ICML, 2017.
- PÉREZ, P.; GANGNET, M.; BLAKE, A. Poisson Image Editing. ACM SIGGRAPH, 2003.
- GUDBJARTSSON, H.; PATZ, S. The Rician Distribution of Noisy MRI Data. Magnetic Resonance in Medicine, v. 34, n. 6, 1995.
- THEMEFISHER. Dtox — template HTML (Bootstrap 4) utilizado como base da identidade visual e da interface. themefisher.com.